Tor CGO 暗号状態のメモリフォレンジック

回路所有関係の復元と TLS・CGO 状態を用いた保存通信の復号

Hiro Nakanishi

独立研究者

quantumsity@protonmail.com

概要

Tor の Counter Galois Onion（CGO）は、ホップごと・方向ごとに暗号状態を更新する。インシデント調査で Tor クライアントのメモリを取得できても、暗号状態らしいバイト列だけでは、稼働回路への所属、入口からのホップ順、取得時点での利用を立証できない。本研究は、C Tor のコアダンプと対応する DWARF 情報を用い、Tor 自身の所有リストから回路を確定し、`cpath` から入口起点の順序を復元する手法を提案する。さらに、回路が保持する入口 TLS 接続をたどり、TLS 1.3 秘密、レコード番号、Tor 回路 ID、CGO 状態を結合することで、取得直後に記録された保存 PCAP 内のリレーセルを TLS 層から CGO 層までオフラインで復号する。Linux arm64 上の複数の Tor 0.4.9 系ビルドでは、独立 12 プロセスの制御下 144/144 回路と 432/432 ホップを復元した。コアダンプ内の状態と取得後のパケットが対応する条件では、Tor と隔離網を毎回作り直した 10/10 独立起動で復号に成功し、各回で本稿が定義する評価段階 E1 から E5 までを連続して検証した。さらに通信を一時停止せず、停止時点をセルや TLS レコードの境界に合わせなかった 6/6 独立起動でも、異なる TLS レコード番号と宛先ホップについて E5 へ到達した。Control Protocol 由来の正解回路 ID を使わず対象回路の全セル候補を列挙し、TLS レコード認証と CGO 認証により取得直後のセルを選んだ。CGO タグを除く 493/493 バイトが評価用リレーメッセージ本体と一致した。この本体は Tor のリレーメッセージであり、HTTPS など上位層の暗号を自動的に解除するものではない。Tor Project が Debian 12 向けに配布するストリップ済み Tor 0.4.9.17 と、対応する外部デバッグシンボルを用いた 3/3 独立起動でも、36/36 回路と 108/108 ホップを復元した。一方、現行状態による直前セルの復号は 3/3 で失敗し、活動中に端点を順次取得した場合は 2.30–3.17 秒の差で状態一致が 0/36 となった。したがって本手法は、端点メモリを取得できる調査者に、回路、経路、暗号方式、取得後の通信内容の対応関係を示すが、過去通信の包括的復号や遠隔からの Tor 匿名性破壊を与えない。

キーワード：Tor、Counter Galois Onion、メモリフォレンジック、コアダンプ、TLS、オフライン復号

Memory Forensics of Tor CGO Cryptographic State: Recovering Circuit Ownership and Decrypting Stored Traffic with TLS and CGO State

Abstract. Counter Galois Onion (CGO) updates cryptographic state for every circuit hop and direction. Finding key-like bytes in a Tor client memory image does not establish which live circuit owned them, which hop was first, or whether the state was active at acquisition time. We present a forensic method that starts from the ownership relationships maintained by C Tor. The method resolves Tor's circuit-owner list using matching DWARF information, recovers entry-oriented hop order from `cpath`, and links the entry TLS object to captured TLS records, Tor circuit identifiers, and per-hop CGO state. Across twelve independent processes covering four Tor 0.4.9 build conditions on Linux arm64, the analyzer recovered 144/144 controlled circuits and 432/432 established CGO hops. Across three further independent processes using the stripped Tor 0.4.9.17 package and matching external debug symbols distributed by the Tor Project for Debian 12, the analyzer recovered 36/36 circuits and 108/108 established CGO hops. In ten fresh, independent Tor runs, an E3 circuit list derived from Tor's ownership relationships constrained both TLS and CGO extraction. A client core and packet capture spanning its acquisition point were then sufficient to authenticate and decrypt TLS 1.3 records, enumerate all cells without a Control Protocol truth identifier, select the unique CGO-authenticated relay cell, recognize its destination hop, and recover all 493 non-tag bytes of the relay body. Six further independent runs succeeded without pausing the application stream or selecting a cell boundary, across six distinct TLS record sequence numbers and destination hops one and two. The current state failed to decrypt the immediately preceding cell in 3/3 runs, and sequential endpoint acquisition during traffic preserved circuit links but lost all 36/36 instantaneous state matches. The contribution is therefore a method for verifying the correspondence among endpoint memory, Tor circuits, and post-acquisition traffic, not a remote cryptanalytic break or general deanonymization attack.

Keywords: Tor, Counter Galois Onion, memory forensics, core dump, TLS, offline decryption

1　はじめに

Tor は、複数のリレーを通る回路と多層の暗号処理によって、通信元と宛先の結び付きを困難にする [1]。この性質は通常の通信保護には有効である。一方、侵害された端末を調べる場面では、調査者が取得したメモリのどこに、どの回路の状態があり、その状態から何を復元できるかが問題になる。

CGO は認証付きの onion encryption を Tor へ導入する方式であり、セル処理に伴って鍵、nonce、累積状態 T' を更新する [2, 3]。更新性は、単純な鍵検索を不十分にする。メモリ上に CGO 構造体を発見しても、それが開いた回路に属するのか、どのホップを表すのか、どの時点のセルへ適用できるのかは分からないからである。

本研究は、次の問いに答える。

> **研究質問：**通常動作中の C Tor クライアントから取得したコアダンプと関連証拠を用いて、どの条件の下で、CGO/TOR1 の回路状態と通信内容をどの範囲までオフラインで復元できるか。

本研究の中心主張　コアダンプに残る TLS 書込みレコード番号と、保存 PCAP 内で認証できる取得後最初のレコード番号が一致する場合、回路所属、入口起点のホップ順、入口 TLS、ホップ別 CGO 状態を対応付け、そのレコード内で CGO 認証に成功するリレーセル本文を復元できる。これは CGO そのものを破る攻撃ではなく、端点メモリ取得後に成立するフォレンジック手法である。

第 2 節に貢献、第 3 節に CGO と調査対象、第 4 節に先行研究との差を示す。第 5 節は証拠に基づく提案手法、第 6 節は評価実装、第 7 節は実験設計、第 8 節は結果を扱う。第 9 節で抽出状態が与える攻撃能力の境界を検証し、第 10 節と第 11 節に実務上の意味と限界を示す。

2　本研究の貢献

本研究の貢献は、次の四点に要約できる。

C1. 回路所有関係を起点とする静的メモリ解析手法。鍵や構造体のパターン走査ではなく、Tor 自身の所有リストからクライアント起点回路（origin circuit）を列挙する。`origin_circuit_t.cpath` を入口とし、マジック値、リスト添字、双方向リンク、閉路を検査することで、候補の回路所属と入口起点のホップ順を同時に検証する。複数バージョン・最適化条件の独立 12 プロセスで、制御下 144/144 回路と 432/432 ホップを復元した。配布パッケージ版の独立 3 プロセスでも、36/36 回路と 108/108 ホップを復元した。

C2. 回路と TLS・CGO 状態の対応関係。回路 ID、ストリーム宛先、入口チャネル、ホップ別 CGO/TOR1、方向別 CGO 状態を、Tor 構造体間のポインタ関係に沿って対応付ける。これにより、メモリ内の鍵らしい値を発見するだけの手法と異なり、「どの回路の、どのホップの、どの方向の状態か」を説明できる。

C3. TLS レコードと CGO 層を順に処理する保存通信の復号。所有回路から入口 OpenSSL オブジェクトを特定し、TLS 1.3 通信秘密と書込みレコード番号を取得する。E3 を通過した全ホップ CGO 回路だけを後段へ渡し、PCAP の TLS 認証復号、全 Tor セル候補の列挙、CGO 層処理を順に行う。通信を一時停止した 10/10 独立起動に加え、通信を止めず停止時点をセル境界に合わせなかった 6/6 独立起動で E1–E5 を連続して検証し、宛先ホップを認識してリレー本文 493/493 バイトを復元した。Control Protocol や動的計測は復号処理へ渡さず、処理終了後の採点だけに用いた。

C4. 成功条件と誤認境界の定量化。構築中の未初期化列挙値、欠損ページ、誤った実行ファイル、誤鍵、取得時点のずれ、直前セル、既知の後方向再同期攻撃を負の対照として評価した。その結果、現行状態はコアダンプ取得後の次セルには使えるが、過去通信の包括的復号、遠隔攻撃、一般的な匿名性破壊を意味しないことを示した。

以上の貢献は CGO の新しい暗号解読ではない。端点メモリを取得できるインシデント調査において、暗号状態を回路と時点へ帰属させ、保存通信まで検証可能にするフォレンジック技術である。

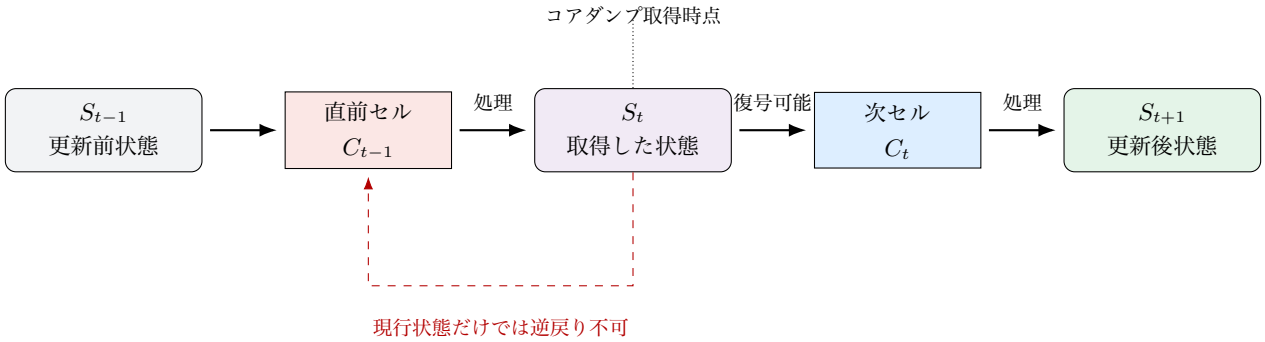


図 1　CGO 状態の時間境界。現行状態は同期した次セルには使えるが、更新前状態を失った直前セルには一般に使えない。

3　背景

3.1　Tor 回路とリレーセル

Tor クライアントは通常、入口、中継、出口の順にリレーを選び、回路を構築する。クライアントから送る前方向（forward）セルには、出口側から入口側へ暗号層を重ねる。受信する後方向（backward）セルは逆の方向に処理される。セルは入口との TLS 接続で運ばれるため、ネットワーク上で観測したパケットは、TLS 層と onion 層の双方で保護されている。

C Tor の origin circuit は、回路 ID、入口チャネル、ストリーム、経路を保持する [4]。経路は `crypt_path_t` の循環双方向リストである。重要なのは、リスト内の最小アドレスが入口なのではなく、`origin_circuit_t.cpath` が入口ホップを指す点である。したがってアドレス順の推測は不要であり、所有構造の意味論から向きを決められる。

3.2　CGO の状態と時間方向

Proposal 359 は、リレーセルの 509 バイトを処理する CGO を定義する [2]。各ホップ・各方向は、疑似乱数関数用の鍵 K、nonce N、認証に用いる累積値 T' などを保持する。CGO はセルを処理するたびに状態を更新する。概念的には、状態 S_i とセル C_i から出力と次状態を得る。

CGO は C Tor 0.4.9.5 でクライアントとリレーが交渉できる安定版機能として公開されたが、Proposal 359 自体の状態は本稿執筆時点で Open である [5, 2]。また 0.4.9.3-alpha から 0.4.9.11 には、最終ホップ以外で輻輳制御を有効にせず、その結果 CGO を交渉しない実装不具合があった。0.4.9.12 は、対応する全ホップで輻輳制御と CGO を交渉するよう修正した [6]。したがって、方式の混在はリレー能力だけでなく、クライアント版と回路構築条件にも依存する。本研究は enum 値を仕様上の意図として読むのではなく、取得時点に実体化したホップ別 context を報告する。

$$(C_i', S_{i+1}) = \mathrm{CGO}(C_i, S_i). \tag{1}$$

スナップショットが保持するのは取得時点の S_t である。S_t から、同じ状態を起点とする将来セルを逐次処理できる。一方、更新前の S_{t-1} が残っていなければ、直前セル C_{t-1} を復号できない。この時間方向は、一般的なディスク復号鍵の回収とは異なる。

Tor Control Protocol は、稼働中の Tor へ問い合わせ、回路 ID、経路、stream 状態などを取得できる [7]。しかし、ホップ別の CGO/TOR1 実体、OpenSSL 内部の TLS traffic secret、方向別 CGO 状態は公開しない。また、プロセス終了後のコアダンプには接続できない。Control Protocol の記録は評価用の正解に限定し、解析入力から除外する。

4　先行研究

4.1　Onion encryption と CGO

初期の Tor は、低遅延のオニオンルーティングを実用化した一方、リレー暗号の改変耐性には形式的な課題があった [1]。Degabriele と Stam は onion encryption を形式化し、Tor の relay cryptography を安全性定義の観点から分析した [8]。その後、Degabriele らは、認証と高速処理を両立する CGO を提案し、構成と性能を示した [3]。さらに、leaky pipes、関連データ、部分的前方秘匿性を含む安全性モデルが検討された [9]。

これらの研究は暗号方式の安全性と状態遷移を扱うが、取得した実プロセスメモリ内で、状態を稼働回路へ帰属させる方法は扱わない。Proposal 359 も実装が保持すべき状態を定義するが、フォレンジック取得、ページ欠損、ABI 差、TLS との証拠結合は対象外である [2]。

4.2　Tor・暗号鍵・TLS のメモリフォレンジック

Alfosail と Norris は、Tor クライアントのメモリ artefact を対象とする調査ワークフローを提案した [10]。Muir らは Tor Browser Bundle を監査し、Windows ホストに残る利用痕跡を評価した [11]。Arshad らは Windows 10 と Android 10 で、レジストリ、ストレージ、メモリ、Zram に残る痕跡を比較した [12]。これらは Tor 利用の立証や端末アーティファクトの収集に有用である。ただし、C Tor 内部の回路所有関係、入口起点の暗号経路、ホップ別 CGO 状態、TLS から CGO までの連続した復号は研究対象としていない。

Tor Project は、メモリ中の統計値がプライバシーへ与える影響も分析している [13]。これはメモリ内情報を最小化する設計上の研究であり、クライアント回路の暗号状態を取得後に解釈する本研究とは目的が異なる。

暗号鍵を RAM から探す研究では、鍵スケジュールの構造を手掛かりに AES などの鍵を識別できる [14]。Cold boot 研究は、劣化した RAM から鍵を再構成できることを示した [15]。これらの成果は、暗号状態がメモリ証拠になり得ることを示す。しかし、鍵が見つかっただけでは、Tor のどの回路、どのホップ、どの方向がその鍵を所有するか分からない。本研究は鍵探索を起点にせず、Tor の所有関係を起点に暗号状態へ到達する。

X-Ray-TLS は、プロセスメモリから TLS 1.2、TLS 1.3、QUIC のセッション秘密を抽出し、通信を透過的に復号する [16]。Baier らは、TLS 鍵素材のメモリ内識別・抽出手法を体系化し、実装差、TLS 1.3、取得後解析の課題を整理した [17]。さらに、規格固有文字列から鍵導出関数を見つけ、主要 TLS 実装へフックする方法も提案されている [18]。これらは、本研究が TLS 秘密の抽出自体を新規性として主張できないことを示す重要な隣接研究である。

しかし、入口リンクの TLS 平文を得ても、その中のリレーセルはホップ別の CGO 層で保護されている。また、TLS 秘密だけからは、Tor のどの所有回路、入口起点のどのホップ、どの取得時点の CGO 状態を適用すべきか決まらない。本研究の差分は、TLS 秘密抽出ではなく、Tor 構造体間のポインタ関係を起点として TLS オブジェクト、リンク回路 ID、ホップ別 CGO 状態、取得時点を対応付ける点にある。表 1 に調査上の差を示す。

表 1　関連手法が解決する調査上の範囲

手法	Tor 所有回路	入口順	TLS 復号	CGO 復号	時点結合
Tor 端末痕跡研究 [10, 11]	–	–	–	–	–
一般鍵探索 [14]	–	–	–	–	–
X-Ray-TLS [16]	–	–	✓	–	–
TLS 鍵調査 [17, 18]	–	–	✓	–	–
本手法	✓	✓	✓	✓	✓

実務では、本手法より軽い方法で調査目的を満たせる場合がある。DFRWS EU 2026 の演習は、端末内の localhost

で Tor へ入る前の通信を取得すれば、暗号化された Tor トンネルの内側に相当する情報を解析できることを示している [19]。これは事前に loopback を取得できる場合には有力であり、アプリケーション内容だけが目的なら本手法より単純である。一方、どの Tor 回路・ホップ・暗号方式がその通信を運んだか、また取得後のコアから何を立証できるかは別の問題である。表 2 に、現場で競合する取得方法との使い分けを示す。

表 2　実務的な代替手法との比較

手法	得られる主な情報	得られない主な情報	適する場面
通常ログ・Control Protocol	稼働中の回路 ID、経路、stream 状態	ホップ別暗号 context、TLS 秘密、終了後の再解析	生存プロセスを即時照会できる
localhost ／ SOCKS 側 PCAP [19]	Tor へ入る前のアプリケーション側通信	onion 回路への帰属、ホップ方式、入口リンク上のセル	事前取得があり内容確認が目的
汎用 TLS 鍵抽出 [16, 17]	入口 TLS 内の Tor セル	CGO 層の relay 本文、所有回路、入口順	TLS 内部だけを確認する
本手法モード A	取得時点の所有回路、入口順、CGO/TOR1 状態	通信本文	core だけが残る事後調査
本手法モード B	上記に加え、時点整合した TLS セルと CGO 認証済み relay 本文	過去の任意通信、上位層暗号の自動解除	core と入口 PCAP が時点対応する

4.3　未解決だった調査上の差分

先行研究を整理すると、未解決の部分は「暗号状態の発見」ではなく「状態の帰属と時点」である。Control Protocol は生きているプロセスの回路を説明できるが、暗号内部状態や取得後解析を提供しない。Tor フォレンジック研究は端末痕跡を扱うが、CGO の状態意味論を扱わない。暗号鍵・TLS フォレンジックは鍵や TLS 平文を得るが、Tor の回路所有関係と CGO 層を復元しない。

本研究の新規性は、この三つを検証可能な対応関係として示した点にある。すなわち、Tor 内部の所有関係から対象回路を確定し、入口からホップを並べ、同じ回路が保持する TLS 状態と CGO 状態を保存 PCAP へ適用する。この組合せにより、構造らしい残骸と稼働状態を区別し、リレー本文まで到達できる。

5　提案手法

5.1　手法の全体像と解析問題

本手法の要点は、メモリ全体から暗号鍵を探すことではない。Tor が管理している所有関係を順方向にたどり、最後に暗号状態へ到達することである。解析順序は、**所有回路の確定、入口とホップ順の確定、暗号方式と初期化状態の確定、TLS および CGO 秘密の抽出、保存通信との照合**である。

この順序には理由がある。ヒープ上には、終了済み回路の残骸、構築中のゼロ初期化領域、解放済みの鍵、別回路の OpenSSL コンテキストが残る可能性がある。暗号値だけを先に発見すると、それがどの回路に属するかを後から証明できない。本手法では、Tor 自身が維持する `global_origin_circuit_list` を探索の起点にする。そこから到達できない構造体や鍵は、値が暗号学的にもっともらしくても稼働回路の証拠として採用しない。

本手法は二つの解析モードを持つ。

モード A：回路状態の静的復元

コアダンプ、対応する実行ファイル、DWARF だけを入力とする。取得時点の所有回路、ライフサイクル、入口起点の経路、ストリーム宛先、ホップ別 CGO/TOR1、方向別 CGO 状態を復元する。対象プロセスは終了済みでもよい。

モード B：保存通信の TLS および CGO 復号

モード A の入力に、コアダンプ取得時点をまたいで入口リンクを記録した PCAP を加える。コア内の TLS 書込みレコード番号と CGO 現行状態に対応する候補を列挙し、TLS 層と CGO 層を順に処理する。過去の任意セルを復号するモードではない。

モード B は、コアダンプを取得して Tor を再開できる場合に適する。クラッシュ時のコアダンプしかなく、その後の通信が存在しない場合は、原則としてモード A までとなる。更新前状態の別コピーが残っていない限り、現行 CGO 状態からコアダンプ取得前のセルへ戻れないためである。

■脅威モデルと信頼境界。 調査者は対象端末を取得する正当な権限を持ち、取得器、解析器、ハッシュ計算環境を信頼できるものとする。コアダンプと PCAP は取得後に改変されず、候補 ELF/DWARF はコア写像との互換性検査を通り、OpenSSL 内部プロファイルは対象ビルドで検証済みであることを前提とする。対象プロセスが侵害されていても、残存構造、未初期化領域、部分欠損は検査対象に含める。一方、取得前に任意メモリ書換えが可能な攻撃者が、Tor の所有リスト、回路構造、暗号状態、PCAP を相互整合するよう意図的に偽造するアンチフォレンジックは対象外である。

E1–E5 は、本稿が処理条件を明確にするために定義した評価段階であり、Tor 仕様の用語ではない。E1–E3 が示すのは、取得物内部で Tor の所有関係と状態が整合することであり、取得手続そのものの法的な証拠保全を代替しない。E4 の TLS AEAD と E5 の CGO 認証タグは、抽出秘密の下で対象バイト列が認証されることを示すが、その通信を特定人物が意図したことや、端末外の送信元帰属までは示さない。したがって本手法の出力は、取得時点、入力ハッシュ、成立した評価段階、未成立理由とともに解釈する。

本手法への必須入力は、時点 t に取得した Tor プロセスのコアダンプ M_t と、M_t を生成した実行ファイルに対応する型情報 B である。コアダンプは、プロセスの仮想メモリとレジスタを保存した取得後解析用のスナップショットである。候補 ELF の Build ID を記録し、コアの `NT_FILE` 対応表が示す主実行ファイル領域と候補 ELF の対応バイト列が、第 6 節で定める 75% 以上一致した場合だけ B を採用する。保存通信を復号する場合だけ、時点 t をまたいでクライアントと入口リレーの間を記録したパケットキャプチャ P を追加する。

解析単位は origin circuit である。回路 c について、本手法は次の結果記録 R_c を生成する。

$$R_c = (I_c, L_c, H_c, A_c, Q_c, U_c, Z_c, E_c). \tag{2}$$

I_c は Tor 内部の回路識別子と入口リンク上の回路 ID、L_c は構築中・確立・終了予約済みなどのライフサイクル、$H_c = (h_0, \ldots, h_{n-1})$ は入口から並べたホップ列である。A_c は回路が所有するストリームと接続先、Q_c は各ホップ・各方向の暗号方式と利用痕跡、U_c は TLS レコードと CGO 状態の時点対応、Z_c は復元できた Tor セルまたはリレー本文、E_c は後述する証拠段階である。入力不足や検証失敗があったフィールドは、推測値ではなく「復元不能」と理由を記録する。

Proposal 359 で定義される CGO の一方向状態を、実装上のアドレスではなく次の論理値として表す [2]。

$$S_{c,d,h}(t) = (K_J, H_J, K_S, H_S, N, T'). \tag{3}$$

c は回路、d は前方向（forward）または後方向（backward）、h はホップ、t は取得時点である。K_J と K_S は二つの AES-128 鍵、H_J と H_S は二つの POLYVAL 鍵、N はノンス、T' はセル処理で更新される累積状態である。各成分は 16 バイトであり、一方向につき合計 96 バイトとなる。解析に必要なのは、この論理値を回路・方向・ホップへ帰属させることである。回路帰属と入口順序の判定はメモリ内のアドレス順や固定オフセットに依存しない。一方、OpenSSL 内部から秘密値を抽出する処理は、後述するバージョン別プロファイルに依存する。

5.2 所有関係から通信復元へ進む処理手順

手法を、コアダンプだけを用いる回路状態復元（アルゴリズム 1）と、保存 PCAP を加える通信復元（アルゴリズム 2）に分けて示す。各処理は、直前までに検証済みとなった証拠だけを受け取る。後段で秘密値や平文らしい値が得られても、前段の検証をさかのぼって成立させない。

所有リスト O_t の位置 j にある候補 c について、E1 と E2 を次のように分離する。

$$\mathrm{Member}(c, j) = [O_t[j] = c], \tag{4}$$

$$\mathrm{Structured}(c, j) = \mathrm{Member}(c, j) \wedge [\mathrm{idx}(c) = j] \wedge [\mathrm{magic}(c) = M_{\mathrm{origin}}]. \tag{5}$$

アルゴリズム中の VerifyExecutable は、候補 ELF の Build ID を記録し、コアダンプの `NT_FILE` が示す主実行ファイル領域と ELF の対応バイト列を照合する。RecoverOwnedMetadata は回路識別子、ライフサイクル、所有ストリームを復元する。RecoverCryptoState は各ホップの方式識別子、前方向と後方向の暗号コンテキスト、式 (3) の CGO 状態を復元する。$T' \neq 0$ は、その方向で状態遷移が起きた利用痕跡として別に記録する。

AuthenticateExactSequence は、コアダンプに保存された書込みレコード番号だけを用い、TLS の関連データ付き認証暗号（Authenticated Encryption with Associated Data: AEAD）を検証する。EnumerateRelayCells は、その認証済みレコードに含まれる対象回路の全セルを返す。Control Protocol 由来の正解回路 ID で一件へ絞らない。ApplyCGOFromSnapshot は各候補を同じ取得時点状態の複製へ入口順に適用し、CGO 認証タグを認識したホップと、タグを除く 493 バイトのリレー本文を返す [2]。E1 は所有リストの全要素を読め、Member が成立した段階である。magic、逆引き添字、経路閉路まで成立した場合だけ E2 へ進む。この分離により、構造破損があっても「Tor がポインタを所有していた」という下位証拠を残せる。入口からのホップ列は、$h_0 = c.\mathrm{cpath}$、$h_{i+1} = \mathrm{next}(h_i)$ として復元する。各ホップで前後リンクの相互一致を検査し、列が h_0 へ戻る場合だけ閉路を受理する。

保存 PCAP を処理する場合、TLS 1.3 の traffic secret を S、レコード番号を i とし、RFC 8446 に従って鍵 k、初期化ベクトル v、nonce n_i を導出する [20]。ハッシュ $\mathcal{H}$ と鍵長 ℓ は cipher suite から決まり、本評価の `TLS_AES_256_GCM_SHA384` では $\mathcal{H} = \mathrm{SHA384}$、$\ell = 32$ である。

$$k = \mathrm{HKDF\text{-}Expand\text{-}Label}_{\mathcal{H}}(S, \mathrm{key}, \ell), \tag{6}$$

$$v = \mathrm{HKDF\text{-}Expand\text{-}Label}_{\mathcal{H}}(S, \mathrm{iv}, 12), \tag{7}$$

$$n_i = v \oplus \mathrm{I2OSP}_{12}(i). \tag{8}$$

アルゴリズム 1 所有関係を起点とする回路状態復元

入力： コアダンプ M_t、候補実行ファイルと型情報 B
出力： 回路ごとの結果記録集合 $\mathcal{R} = \{R_c\}$

$\mathcal{R} \leftarrow \emptyset$
if VerifyExecutable(M_t, B) が不成立 **then**
  返す {error(実行ファイル不一致)}
end if
$\Lambda \leftarrow$ ResolveTypes(B)
$O_t \leftarrow$ ReadOwnerList(M_t, Λ)
if O_t の領域、要素数、容量、全要素を検証できない **then**
  返す {incomplete(所有リスト取得不能)}
end if
for $j \leftarrow 0$ **to** $|O_t| - 1$ **do**
  $(I_c, L_c, H_c, A_c, Q_c, U_c, Z_c, E_c) \leftarrow (\perp, \perp, \perp, \perp, \perp, \perp, \perp, \text{E0})$
  $c \leftarrow O_t[j]$; $E_c \leftarrow$ E1
  $(I_c, L_c, A_c) \leftarrow$ RecoverOwnedMetadata(c)
  $H_c \leftarrow$ WalkFromEntry$(c.\text{cpath})$
  if Structured(c, j) かつ H_c が双方向リンクと閉路の検査を通過 **then**
    $E_c \leftarrow$ E2
    $Q_c \leftarrow$ RecoverCryptoState(c, H_c)
    if 回路が活動中かつ確立状態で、全ホップの方式別暗号コンテキストが整合 **then**
      $E_c \leftarrow$ E3
    end if
  end if
  $R_c \leftarrow (I_c, L_c, H_c, A_c, Q_c, U_c, Z_c, E_c)$
  $\mathcal{R} \leftarrow \mathcal{R} \cup \{R_c\}$
end for
返す $\mathcal{R}$

アルゴリズム 2 TLS レコードと CGO 層を順に処理する保存通信の復号

入力： $\mathcal{R}$、コアダンプ M_t、入口リンクを含む保存 PCAP P
出力： E4 または E5 へ更新した $\mathcal{R}$

for all $R_c \in \mathcal{R}$ かつ $E_c =$ E3 かつ全ホップが CGO **do**
  $T_c \leftarrow$ RecoverEntryTLS(M_t, c)
  if T_c の TLS 版、役割、暗号スイート、秘密長、回路帰属を検証できる **then**
    $\mathcal{D}_c \leftarrow$ AuthenticateExactSequence$(P, T_c, T_c.\text{write_seq})$
    $\mathcal{C}_c \leftarrow$ EnumerateRelayCells$(\mathcal{D}_c, I_c.\text{link_circid})$
    if $\mathcal{C}_c \neq \emptyset$ **then**
      $E_c \leftarrow$ E4; $U_c \leftarrow (T_c.\text{write_seq}, \mathcal{D}_c)$
      $\mathcal{A}_c \leftarrow \emptyset$
      for all $q \in \mathcal{C}_c$ を PCAP 内の順に走査 **do**
        $(h, z) \leftarrow$ ApplyCGOFromSnapshot(q, H_c, Q_c)
        if h で CGO 認証タグを認識 **then**
          $\mathcal{A}_c \leftarrow \mathcal{A}_c \cup \{(q, h, z)\}$
        end if
      end for
      if $|\mathcal{A}_c| = 1$ **then**
        $E_c \leftarrow$ E5; $Z_c \leftarrow$ only$(\mathcal{A}_c)$
      end if
    end if
  end if
end for
返す $\mathcal{R}$

全候補で認証に失敗した場合、または複数候補が認証された場合は E4 に留まり、本文を出力しない。この手順では、構造検査、TLS 認証、CGO 認証が、相互に異なる状態と鍵に基づく棄却点になる。統計的独立性は仮定しない。偶然もっともらしい構造、誤った TLS 秘密、誤った CGO 状態が平文復元まで無検証で連鎖することを防ぎ、各段階の入力、検査結果、停止理由を保存する。

5.3 証拠段階と判定条件

対象は、調査権限のある端末で動作する C Tor クライアントである。入力は、(1) ELF コアダンプ、(2) Build ID を記録し、コア内の実行ファイル写像との互換性を検証した実行ファイルと DWARF である。証拠段階 E4 または E5 を判定する場合は、(3) 取得時点をまたぐ入口リンクの PCAP も必要となる。解析は取得後に行い、動的デバッガや特別な識別子を対象プロセスへ埋め込まない。

復元成功は、単に候補が出たことではなく、次の検査を通過したことと定義した。

- 回路が Tor 自身の origin circuit 所有リストに含まれる。
- list index、magic、ポインタ範囲、`next/prev`、閉路が整合する。
- lifecycle と暗号状態の初期化が整合する。
- PCAP 復号では TLS AEAD 認証、Tor 回路 ID、CGO タグ認識を順に満たす。評価実験では、復元本文を別に取得した正解平文と比較する。

追加計測から得た Control Protocol 記録、動的フックのセル、合成 identity は評価専用の正解であり、解析入力から除外する。

解析結果の証拠強度を表 3 の五段階で表す。上位段階は下位段階の成立を前提とする。例えば、CGO 認証タグを認識しても、所有リストと cpath の検査を通っていなければレベル E5 にはしない。

表 3　解析結果の証拠段階

段階	成立条件	調査上言えること
E1	所有リストを解決し要素を列挙	Tor が取得時点に回路を所有していた
E2	magic、index、双方向リンク、閉路が整合	入口起点の経路構造が復元できた
E3	open 状態と暗号 context 初期化が整合	ホップ別の実体化方式を確定できた
E4	入口 SSL と TLS record を AEAD 認証	PCAP 内 Tor セルを回路へ結合できた
E5	CGO tag を認識し本文を取得	対象 hop の relay 本文を復元できた

E1–E3 は core だけで成立し、E4–E5 には時点が整合する PCAP が必要である。途中段階で失敗しても、成立済みの下位段階は破棄しない。例えば TLS profile が未対応でも、所有回路と経路の復元結果は E2 として報告できる。

取得したメモリを時点 t のスナップショット M_t とする。M_t には、稼働中の構造、終了後の残骸、未初期化領域、無関係な暗号 context が混在し得る。本手法は、構造体の形や鍵のエントロピーを根拠に候補を集めるのではなく、Tor が時点 t に所有していた origin circuit の集合 O_t を最初に求める。

回路 c を調査対象として受理する条件は、$c \in O_t$ であり、かつ所有リスト上の位置と c 自身が持つ所属情報が一致することである。この条件を満たさないメモリ断片は、後続のフィールドがもっともらしくても「稼働回路」とは扱わない。これにより、解放済み構造や偶然整合したバイト列を、現在の回路へ誤帰属させる危険を抑える。

この段階で立証するのは、回路が取得時点に Tor の管理下にあったという事実である。通信が成功したこと、特定方式が使われたこと、セル本文を復号できることまでは主張しない。後続の各段階は、この所有証拠へ追加の条件を積み重ねる。

origin circuit の経路は循環リストである。循環リストだけを見れば、任意の要素を先頭にできるため、アドレスの大小や走査で最初に見つかった位置から入口を推定してはならない。本手法は、origin circuit が入口ホップとして保持する参照を p_0 とし、$p_0, p_1, \ldots, p_{n-1}$ を `next` 関係で並べる。

並びを受理するには、各 p_i で前後のリンクが相互に戻り、最後の要素から p_0 へ閉じる必要がある。途中で NULL、別の閉路、読取り不能領域へ達した場合は、回路の存在が E1 で確認できていても経路復元を E2 へ進めない。

この方法は「入口らしさ」を統計的に推定するものではない。Tor 実装が持つ所有参照の意味を用いて向きを確定する。そのため、同じ三つのホップを得た場合でも、順序が逆または回転していれば成功とは数えない。

暗号状態について、次の三つを区別する。

交渉意図

対向リレーの能力とハンドシェイク情報から、CGO を選択しようとしたことを示す。これは暗号 context が完成したことを保証しない。

実体化方式

ホップが確立状態であり、方式識別子と必要な前方向・後方向の暗号コンテキストが整合することを示す。この条件が成立した場合だけ CGO または TOR1 を確定する。

方向別の利用痕跡

CGO の累積状態 T' が初期値から変化したことを示す。これはその方向で状態遷移が起きた証拠であるが、特定 stream の通信、正確なセル数、ユーザ行為までは示さない。

この分離は building 中の回路に重要である。ゼロ初期化された enum が TOR1 と同じ値でも、context が未初期化なら「TOR1 確立」とは判定しない。また、終了後に鍵断片が残っていても、所有集合 O_t に回路がなければ現在の稼働回路へ結び付けない。

5.4 取得時点の暗号状態による通信復元

CGO 状態はセルごとに更新される。時点 t のコアダンプが保持する対象回路・方向・ホップの状態を $S_{c,d,h}(t)$ とする。この状態を適用できるのは、同じ状態を消費する次のセルである。したがって、PCAP 内に同じ回路 ID のセルがあるというだけでは不十分であり、TLS レコード番号、入口リンクの回路 ID、CGO 認証を使って時点を結合する。

モード B では、次の条件をすべて要求する。

1. PCAP がコアダンプ取得時点 t をまたいで入口リンクを記録している。
2. コアダンプ内のクライアント TLS 書込みレコード番号と、PCAP で AEAD 認証できたクライアント側レコードの番号が一致する。
3. そのレコード内で、対象入口回路 ID を持つ全リレーセルを出現順に列挙する。
4. 各候補へ $S_{c,\mathrm{forward},0}(t)$ の複製を入口順に適用し、CGO 認証タグを最初に認識する候補だけを採用する。

同じ TLS レコードに別回路のセルが先に存在しても、対象回路の CGO 状態は進まない。一方、対象回路のセルがコアダンプ取得と PCAP レコードの間に一つでも欠落すれば、後続セルとの状態同期は失われる。したがって、PCAP 欠損を許容して後続セルへ飛ぶ処理は行わない。

壁時計の時刻は補助情報であり、復号対象を決める主根拠ではない。コアダンプ内の TLS 書込みレコード番号、入口リンクの回路 ID、CGO 認証という三つの証拠を用いる。対象回路の候補は認証済み TLS レコードからすべて列挙し、CGO 認証に成功する候補が一件だけの場合に確定する。これにより、取得ホストの時計誤差や、単なる出現順だけで別レコード・別セルを選ぶことを避ける。

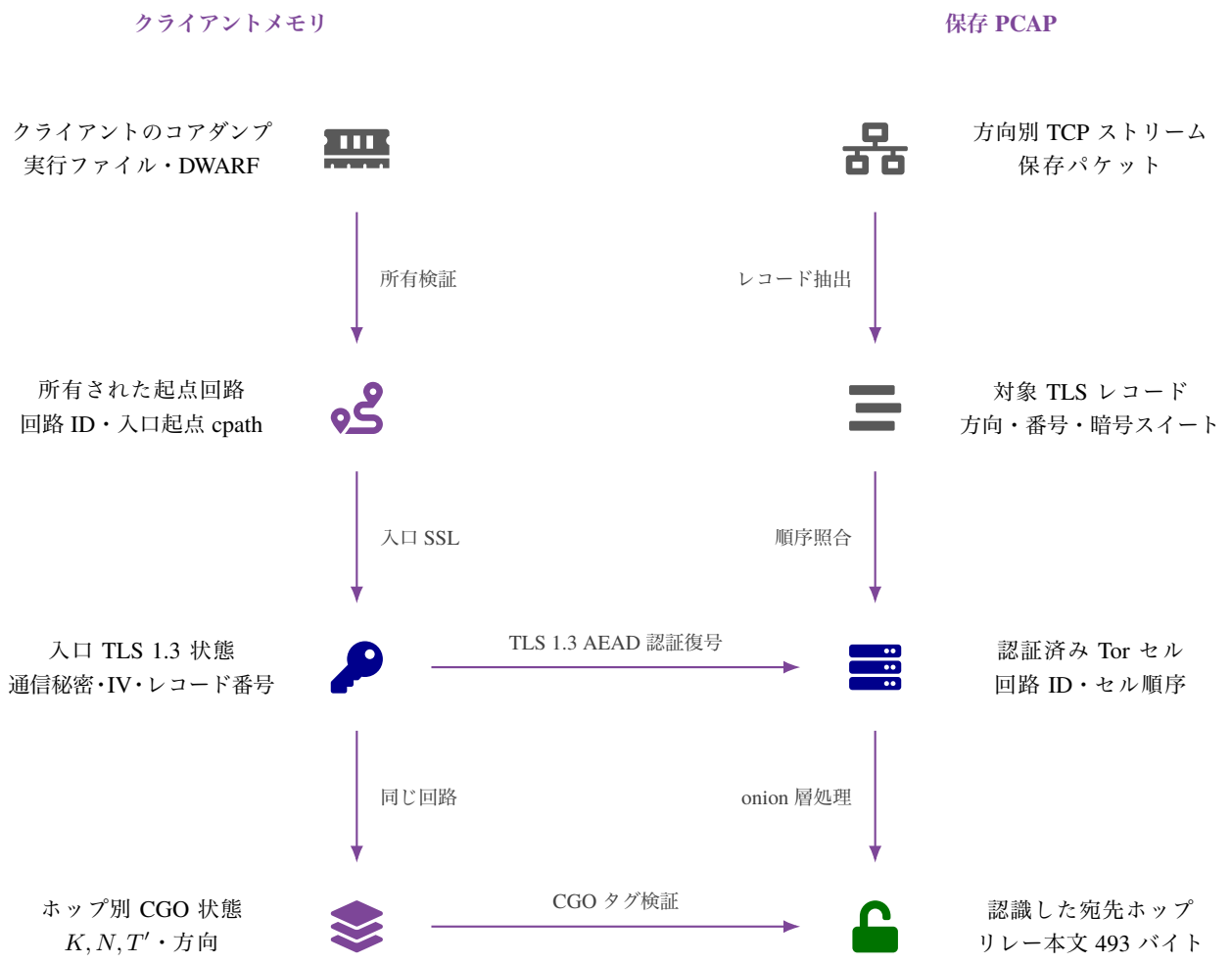


図 2　保存 PCAP とクライアントメモリの対応関係。各段階が次段階の対象と検証条件を与える。

PCAP からリレー本文までには、相互に異なる鍵と状態を用いる二つの認証判定がある。第一は TLS 1.3 の AEAD 認証タグである。正しい通信秘密、nonce、レコード番号、TLS ヘッダがそろわなければレコードを採用しない。第二は CGO 認証タグである。入口から順に各ホップの状態を適用し、タグを認識したホップを宛先とする。両判定の統計的独立性は仮定しない。

TLS 認証だけが成功した場合、PCAP 内の Tor セルを入口 link へ結び付けた E4 までを報告できる。しかし、CGO tag をどの hop も認識しなければ、状態ずれ、セル欠損、方向の誤り、未対応実装のいずれかであり、本文を出力しない。CGO tag を認識した場合だけ、tag 部分を除く 493 バイトを E5 の relay 本文として扱う。

評価用平文との一致は手法の成立条件ではなく、実験で正しさを測るための外部判定である。実際の調査では正解平文がなくても、TLS AEAD と CGO tag という二段階の認証が復号結果の内部検証になる。

本手法は、成功した最上位の証拠段階と、そこで用いた回路・方向・時点を結果として返す。E2 なら「取得時点に所有されていた回路と入口起点経路」、E3 なら「各 hop で確立した暗号方式」、E4 なら「保存 PCAP 中の認証済み Tor セル」、E5 なら「認証された宛先 hop と relay 本文」である。

失敗は否定証明ではない。所有リストを読めなかった 0 件は「回路不存在」を意味せず、CGO 認証タグの不一致は「通信がなかった」ことを意味しない。各段階で必要な証拠が欠けたと報告する。この保守的な判定規則を、残存構造や欠損ページによる誤認を避ける中心原則とする。

6　実装

評価実装は、GDB の Python API でコアダンプを読み、DWARF 型情報と検証済みプロファイルに基づいて OpenSSL 内部状態を解釈する [21, 22, 23]。環境に依存するのは、型情報の解決方法と OpenSSL 内部オフセットである。回路

所有、入口順、時点整合、二段階認証の判定条件は、これらの実装選択に依存しない。

6.1 解析パイプラインと実行ファイルの同定

解析は表 4 の五段階に分けた。各段階は、前段の出力に含まれるアドレスまたは識別子だけを次段へ渡す。したがって、後段で暗号鍵らしい値が見つかっても、所有回路まで逆向きに結び付かない候補は採用しない。

表 4　解析パイプラインと段階別の検証条件

段階	主な処理	次段へ渡す証拠
P1 型解決	Build ID を記録し、コア写像、DWARF、PIE バイアスを照合	型、フィールド位置、所有リスト位置
P2 回路復元	所有リスト、magic、index、閉路を検証	回路 ID、入口 cpath、入口チャネル
P3 状態抽出	方式、初期化、OpenSSL プロファイルを検証	TLS 秘密、レコード番号、ホップ別 CGO 状態
P4 通信復号	TCP 再構成、TLS AEAD、Tor セル選択	509 バイトのリレーセルペイロード
P5 CGO 処理	入口から各層を処理し認証タグを検証	認識ホップ、493 バイトのリレー本文

保存 core の解析は読み取り専用であり、対象プロセス内の関数呼出しと値の書換えを伴わない。JSON 形式の結果には、回路ごとの所有根拠、向き、構造検査、暗号初期化、エラー、完全性を含める。

候補実行ファイルの SHA-256 と ELF Build ID を記録する。Linux コアは実行ファイルの Build ID を常に直接保持するとは限らないため、`NT_FILE` 対応表から主実行ファイルの写像を求め、コアに収録されたファイル由来バイトと候補 ELF の同一 offset を比較する。`gcore` では clean な実行ページが省略され、再配置・可変データは変化するため、比較可能な 110,592 バイトのうち 75% 以上の一致を互換性条件とした。正しい ELF は 84.51% 一致し、同版 LTO と異版は 8.71%、9.58% で専用エラーになった。この閾値は Build ID の代替となる一般的同一性証明ではなく、本取得形式で誤 ELF を型解決へ渡さないための事前条件である。

通常ビルドでは、`global_origin_circuit_list` の位置と各構造体のフィールド位置を DWARF から得る。このため、回路解析は Tor バージョンや最適化条件ごとの固定オフセット表を必要としない。

LTO では所有リスト変数の DWARF location が optimized out になる場合がある。この場合だけ、同一 ELF のローカルシンボルと、実行時にも解決できる関数 `circuit_get_global_origin_circuit_list` を anchor として PIE load bias を求める。ELF 上の anchor アドレスを A_{elf}、core 内の実行時アドレスを A_{run} とすると、

$$b = A_{\mathrm{run}}(\mathrm{anchor}) - A_{\mathrm{elf}}(\mathrm{anchor}), \qquad A_{\mathrm{run}}(s) = A_{\mathrm{elf}}(s) + b \tag{9}$$

である。候補シンボルが 0 個または複数なら解析を停止する。外部から明示した load bias を使う場合も、対象 ELF 内で owner-list symbol が一意であることを要求する。誤バイナリを推測で補正しない。

6.2 所有回路・経路・ストリームの復元

解析器は DWARF から型、フィールド位置、enum、グローバル変数を解決する。最初に `global_origin_circuit_list` を読み、全要素を列挙できた場合に各要素を E1 とする。その後、各 `origin_circuit_t` の magic と list index を確認する。smartlist について、`num_used` が 0 以上、`capacity` 以下であり、`capacity` が 10^6 以下であることを確認する。範囲外なら列挙しない。要素 i を `origin_circuit_t*` へ cast した後、`base_.magic` が `ORIGIN_CIRCUIT_MAGIC` であることと、構造体内の `global_origin_circuit_list_idx` が i に一致することを検査する。さらに入口起点閉路が成立した要素だけを E2 とする。

各回路では `origin_circuit_t.cpath` を入口ホップとする。以後、最大 32 ホップまで `next` をたどり、各ノードに

ついて次の不変条件を検査する。

$$\texttt{hop.next.prev==hop},\quad \texttt{hop.prev.next==hop},\quad \texttt{hop.magic==CPATH_MAGIC}.$$

既訪問アドレスへ達したとき、そのアドレスが最初の cpath と同じ場合だけ閉路成立とする。NULL、読取り不能、別の既訪問ノード、32 ホップ超過は構造エラーとして記録する。入口は所有参照によって決まり、ホップのアドレス順には依存しない。

回路ごとに、Control Protocol と対応可能な global_identifier、入口リンクの n_circ_id、n_chan のチャネル ID と対向識別子を取得する。これらは探索条件ではなく、後続の TLS オブジェクトと PCAP 内 Tor セルを同じ回路へ結ぶ結合キーになる。

E1–E3 の検査後、所有リストを完全列挙でき、活動中の回路と全ホップが確立状態で、全ホップが CGO である回路だけを E3 manifest へ記録する。manifest には所有リスト添字、回路アドレス、二種類の回路 ID、入口起点のホップアドレス列、nonce と T' のハッシュ、元の構造解析結果の SHA-256 を含める。TLS 秘密抽出器と CGO 秘密抽出器は同じ manifest を必須入力とし、所有リスト要素、回路アドレス、回路 ID、ホップ数、ホップアドレスが一致しなければ停止する。したがって、E1–E3 を通過していない候補が E4–E5 だけを成立させる経路はない。

クライアント起点回路が所有する p_streams から edge_connection_t の単方向リストをたどる。各要素について on_circuit が現在の回路アドレスと一致すること、入口接続の magic、ストリーム ID、cpath_layer を確認する。SOCKS 要求が存在する場合は、ハンドシェイク完了、宛先、ポートを取得する。

ストリームの帰属には、ヒープ全体の SOCKS 構造体ではなく、Tor が保持する p_streams 所有辺を用いる。ユーザ名とパスワードは回路帰属に不要であるため収集対象から除外する。接続元端点は、調査目的上必要な場合に限って記録する。

6.3 CGO と入口 TLS 状態の抽出

各 crypt_path_t から relay_crypto_t へ進み、暗号方式 enum だけでなく初期化済みかを確認する。kind=0 を TOR1、kind=1 を CGO と解釈し、それ以外は未知値とする。building 中のゼロ初期化メモリでは、enum のゼロが TOR1 と同じ値に見える場合がある。そのため、回路と hop が open であり、対応する暗号コンテキストが初期化済みの場合だけ方式を確定する。TOR1 では forward/backward の cipher と digest の 4 ポインタ、CGO では forward/backward の cgo_crypt_t* がすべて非 NULL であることを要求する。extend_info.enable_cgo が得られる場合は、交渉意図と実体化した kind の一致も記録する。

CGO では前方向と後方向を別々に処理し、表 5 の 96 バイトを一方向の現行状態として取得する。Tor 構造体が直接保持する POLYVAL 鍵、ノンス、T' は DWARF で解決する。AES 鍵だけは EVP_CIPHER_CTX から OpenSSL プロバイダーのコンテキストをたどる。

表 5　一方向の CGO 現行状態として取得する成分

成分	長さ	取得元と検査
J 用 AES-128 鍵	16 B	プロバイダーの AES 復号鍵スケジュール
J 用 POLYVAL 鍵	16 B	uiv.j.ku.key.h
S 用 AES-128 鍵	16 B	プロバイダーの AES 暗号鍵スケジュール
S 用 POLYVAL 鍵	16 B	uiv.s.b.h
nonce N	16 B	cgo_crypt_t.nonce
累積状態 T'	16 B	cgo_crypt_t.tprime

OpenSSL 3 の内部には公開 ABI がない。暗号状態の抽出は、対象ライブラリが検証済みの openssl-3.0.22-debian-arm64 プロファイルと一致する場合に限定する。このプロファイルでは EVP_CIPHER_CTX.algctx を

オフセット 168 で読み、プロバイダーの AES 鍵スケジュールを algctx+192 から取得する。AES ラウンド数は鍵スケジュール内のオフセット 240 で読み、AES-128 を示す 10 でなければ拒否する。J の復号鍵スケジュールでは最終ラウンドの位置、S の暗号鍵スケジュールでは最初のラウンドの位置から 16 バイトを取得した。この対応は同じビルドの単体試験が保持する現行鍵と照合して検証した。

algctx==NULL、AES round 数の不一致、各成分が 16 バイトでない場合、読み取り不能ページがある場合は、その方向を復号に使わない。標準出力には状態全体と T' の SHA-256 だけを記録する。生の 96 バイトを証拠として保存する場合は、アクセス権を mode 0600 とする。また、$T' \neq 0$ は、その方向で状態遷移が起きた証拠として扱う。特定ユーザの通信やセル数を示す証拠とは扱わない。

所有確認済み origin circuit から、次の型付きポインタ列をたどる。

$$\texttt{origin_circuit_t} \rightarrow \texttt{base_.n_chan} \rightarrow \texttt{channel_tls_t.conn} \rightarrow \texttt{or_connection_t.tls} \rightarrow \texttt{tor_tls_t.ssl} \rightarrow \texttt{SSL}$$

途中のポインタが NULL なら対象回路には入口 TLS 証拠がないとして停止する。SSL.version が 0x0304、SSL.server と tor_tls_t.isServer がともにクライアント側を示すことを確認する。検証済み OpenSSL プロファイルからクライアント／サーバのアプリケーション通信秘密、読込み／書込み IV、読込み／書込みレコード番号、クライアント乱数を読む。

暗号スイートは通信秘密の長さから推測しない。対象 OpenSSL ソースとディストリビューション設定から SSL.session、SSL_SESSION.cipher、SSL_CIPHER.id の位置を導出し、コアダンプ内の識別子を直接読む。本評価では全対象が 0x1302、すなわち TLS_AES_256_GCM_SHA384 であった。識別子、ハッシュ、秘密長が対応しない場合、または実装していない TLS_CHACHA20_POLY1305_SHA256 の場合は復号へ進まない。

表 6　OpenSSL 3.0.22 Debian arm64 用 TLS プロファイルの主要オフセット

フィールド	オフセット	フィールド	オフセット
SSL.version	0	SSL.server	56
client random	352	SSL.session	2328
client app secret	1860	server app secret	1924
read IV	2128	write IV	2176
read sequence	7416	write sequence	7424
SSL_SESSION.cipher	760	SSL_CIPHER.id	24

通信秘密領域 64 バイトの末尾がゼロであることを確認し、非ゼロ部分を 32 または 48 バイトとして採用する。クライアント／サーバの長さが異なる場合と全ゼロの場合は拒否する。暗号スイート識別子から決まるハッシュと秘密長が実データに一致する場合だけ採用する。レコード番号は OpenSSL レコード層の 8 バイト big-endian 値として読む。

6.4　TLS レコードと CGO 層の復号条件

コアダンプ取得後の保存通信の復号は、図 2 の順に行う。PCAP から Ethernet/IPv4/TCP ペイロードを取り出し、送信元・宛先 IP とポートの四つ組ごとに TCP 番号順で再構成する。同じ番号の再送は最初のペイロードを一つだけ残す。再構成したストリームから 5 バイトの TLS ヘッダとレコード本体を切り出す。

アプリケーション通信秘密から、式 (6) に従って AES-256-GCM の鍵と nonce を導出する。

外側の TLS ヘッダを関連データとし、コアダンプが保持するクライアント側書込みレコード番号を nonce へ用いて GCM 認証タグを検証する。0–64 の総当たりは行わない。正しい TLS 秘密でも別のレコード番号を与えれば、その番号に対応する後続レコードは暗号学的に正しく認証され得る。このため、AEAD だけで時点を判断せず、コアダンプ内番号と取得タイムラインの一致を E4 の必須条件とする。

TLSInnerPlaintext の末尾パディングと内部 content type を除き、アプリケーションデータから Tor セルを探す。対

象セルの wire 形式は、4 バイトの入口リンク回路 ID、1 バイトのコマンド、509 バイトのペイロードである。コマンドは RELAY=3 または RELAY_EARLY=9 に限定した。同じ TLS レコード内に対象回路のセルが複数ある場合、Control Protocol 由来の正解回路 ID や正解セルで一件へ絞らず、出現順に全候補を列挙する。

抽出した各ホップの前方向状態 80 バイトと T' 16 バイトから、Tor 実装と同じ cgo_crypt_t を解析用プロセス内に再構成する。各セル候補について取得時点状態の複製を作り、509 バイトをセルペイロードへ設定し、入口ホップから順に cgo_crypt_relay_forward 相当の処理を適用する。全候補を処理した後、認証タグを認識した候補が厳密に一件である場合だけ、その候補とホップを採用する。認証候補が 0 件または複数件なら状態不一致または曖昧として平文を出力しない。

CGO 認証タグである先頭 16 バイトを除いた 493 バイトをリレーメッセージ本体として出力する。復号用の試験入力には、コアダンプ由来の CGO 状態と PCAP 由来の暗号文だけを格納し、評価用の正解ホップと正解平文を含めない。復号処理が終了した後、別プロセスの評価器だけが、動的フック由来の正解セルと認識ホップ、493 バイトを比較する。この物理的分離により、正解情報を回路選択、TLS 復号、試験入力、CGO 処理へ混入させない。

解析結果には秘密値そのものを含めず、方式、初期化状態、更新有無、検証結果、秘密値の SHA-256 を記録する。リレー本文は標準出力から除外し、証拠内容の取得が調査目的に含まれる場合だけ保存する。

解析結果が 0 件でも、回路が存在しなかったとは限らない。所有リストの smartlist を含むページが欠損すれば、解析器は候補を得られないからである。そこで、必要な ELF セグメント、所有リスト、各回路、各暗号経路、OpenSSL 領域について、読み取りの成否と列挙の完了状態を出力する。所有リストを全要素列挙できた場合は「列挙完了」、一部で読取りエラーが生じた場合は「部分復元」、所有リスト自体を解決できない場合は「取得不能」とする。num_used=0 の場合に言えるのは、取得時点のリストが空であったことだけである。必須ページが欠けた場合は「0 回路」ではなく「復元不完全」と報告する。

個別回路では、構造検査と暗号検査を分離する。所有回路として列挙できても cpath 閉路が壊れていれば、回路メタデータは残し、暗号状態を確定しない。CGO 構造体を読めても OpenSSL プロファイル検査に失敗すれば、方式は CGO、秘密抽出は失敗として報告する。この段階別出力により、単一の欠損を「回路不存在」や「TOR1 利用」と誤って解釈することを防ぐ。

標準 JSON にはアドレス、方式、状態、検査結果、秘密値のハッシュだけを保存する。TLS 秘密、CGO 状態、復号済み TLS レコード、リレー本文を証拠として保存する場合は、生成ファイルのアクセス権を 0600 とする。公開用集計には生の秘密を含めない。

7 実験設計

7.1 隔離環境と対象

第三者通信を含まない Chutney 隔離網を構築し、制御下のクライアントとリレーだけを使用した。使用した Chutney と C Tor のコミットを固定した [24, 4]。中心評価は Linux arm64 上の C Tor 0.4.9.16、OpenSSL 3.0.22 である。一般化評価では C Tor 0.4.9.3-alpha、0.4.9.9、0.4.9.16、-O0/-O2/LTO を用いた。x86_64 ABI は QEMU user-mode の guest core で評価した。これはネイティブ x86_64 評価ではない。実環境への適用可能性を確認する評価では、Tor Project 公式リポジトリが Debian 12 arm64 向けに配布する Tor 0.4.9.17 と tor-dbgsym を用いた [25]。実行ファイルはストリップ済みであり、GNU Build ID が一致する外部デバッグシンボルを型情報として使用した。

0.4.9.3-alpha と 0.4.9.9 には、通常の自動構築回路で最終ホップ以外の CGO 交渉を妨げる既知不具合がある [6]。G1 の制御回路は明示経路を EXTENDCIRCUITpurpose=controller で構築して全ホップ CGO を成立させた解析器評価であり、各版の自然な CGO 普及率を測る実験ではない。この差を確認するため、自動生成された ambient 回路の方式分布も別に集計した。

実験用ストリームの宛先、送信元、識別子は合成値とした。SOCKS 認証情報は解析結果へ出力しない。メモリ、PCAP、正解記録には SHA-256 を付与し、実行環境とソースコードのコミット識別子を記録した。

通信を一時停止する実験の時系列を図 3 に示す。PCAP を先に開始し、ストリームを一時停止してから Tor を SIGSTOP し、コアダンプを取得する。Tor を再開した後にストリームを再開し、入口リンクへ出る次の TLS レコードを保存する。動的フックは評価用セルを別ファイルへ保存するだけであり、復号処理へは渡さない。

これとは別に、停止時点をセルや TLS レコードの境界に合わせない実験を行った。PCAP と継続ストリームを開始し、ストリームを一時停止せず、動的セルフックも使わず、0.11–1.61 秒の異なる遅延後に gcore を実行した。gcore がアドレス空間を複写する間のプロセス停止は、整合したメモリ取得に必要な挙動として残したが、セル処理や TLS レコード境界から停止時点を選ばなかった。2 ホップと 3 ホップを各 3 独立起動とし、復元処理は正解セルを受け取らない。CGO 認証で候補を一意化した後だけ、制御下ストリームへ繰り返し挿入した評価用マーカーを確認した。



図 3　通信を一時停止する実験の時系列。評価記録は復号終了後の比較にだけ用いる。

表 7 に実験群と独立単位をまとめる。件数の異なる評価を同一母集団として合算しない。

表 7　実験群、目的、独立単位

ID	目的と条件	独立単位	規模
G1	0.4.9 系・最適化差の構造復元	Tor プロセス	12 起動、144 回路
G2	ライフサイクルと残存構造	回路／取得時点	確立 12、構築中 8、終了 6
G3	TLS レコードと CGO 層の連続復号	Tor プロセス	10 起動、各 7 候補
G4	端点間の時間差と直前セル	Tor プロセス	3 起動、36 ホップ
G5	実行ファイル・欠損・誤状態負対照	条件	ELF 3 種、欠損 2、誤状態 7,168
G6	既知再同期攻撃の実装境界	Tor プロセス	3 起動
G7	ストリーム非停止・停止時点非選択の復号	Tor プロセス	6 起動、2/3 ホップ
G8	CGO/TOR1 混在回路の後段入力制限	回路	19 回路、うち混在 10
G9	公式 Debian パッケージ版への適用	Tor プロセス	3 起動、36 回路

7.2　評価項目

評価は次の六群に分けた。

1. 所有回路、入口起点パス、CGO/TOR1 混在の正確性。
2. バージョン、最適化、LTO、ABI の差に対する再現性。
3. 確立／構築中／終了後、欠損ページ、誤バイナリに対する誤受理抑制。
4. ストリーム、回路、経路、CGO 状態、次セルの結合。
5. コアダンプと PCAP だけを入力とする TLS レコードおよび CGO 層の復号。
6. 取得時点のずれ、直前セル、誤鍵、既知の後方向再同期攻撃という境界。

回路・ホップは同じプロセス内で相関する。したがって、反復成功率は Tor プロセスの状態を毎回初期化した起動を単位とし、二側 95% Wilson スコア区間を示す。この区間が表すのは固定した実験条件での反復性であり、一般環境での成功率ではない。決定論的に生成した誤状態は確率標本ではないため信頼区間を付けず、観測件数だけを報告する。

8　結果

8.1　検証条件を満たす回路・暗号状態を復元した

初期の実 Tor 評価では、0.4.9.16 単独環境の 20/20 回路、0.4.9.16 と 0.4.8.25 を混在させた環境の 19/19 回路が Control Protocol の回路 ID と一致した。入口起点の全経路も 39/39 で一致した。混在環境の 51 ホップは、CGO 40、TOR1 11 に分類された。0.4.8.25 リレーを含むホップはすべて TOR1 であった。この混在コアを最終処理手順で再解析すると、活動中 E3 回路 19 件のうち全ホップ CGO は 8 件、CGO/TOR1 混在は 10 件、TOR1 のみは 1 件だった。E4–E5 の対象回路一覧は全 CGO 8 件と厳密に一致し、混在回路を CGO 専用復号処理へ渡した件数は 0/10 だった。したがって、混在回路の方式分布は E3 で報告するが、現行の CGO 専用復号処理には渡さない。

より広い評価では、四つの arm64 条件を各 3 回起動し、制御下 144/144 回路と 432/432 ホップを復元した（表 8）。12 プロセスすべてが完全成功し、プロセス単位の 95% Wilson 区間は 75.75–100% である。

表 8　バージョン・最適化差に対する復元結果

条件	完全成功	回路	CGO ホップ
0.4.9.3-alpha, `-O0`	3/3	36/36	108/108
0.4.9.9, `-O0`	3/3	36/36	108/108
0.4.9.16, `-O0`	3/3	36/36	108/108
0.4.9.16, `-O2`	3/3	36/36	108/108
合計	12/12	144/144	432/432

回路・ホップは同一プロセス内で相関するため、独立成功率の単位はプロセスとした。

自己ビルドではない配布パッケージへの適用結果を表 9 に示す。Tor Project 公式リポジトリの Debian 12 arm64 向け Tor 0.4.9.17 はストリップ済みであり、実行ファイルの GNU Build ID は `59de31af...a9f` だった。同じ公式リポジトリの `tor-dbgsym` に含まれる外部デバッグファイルは、この Build ID と一致した。3 回の独立起動で 36/36 回路の所有関係と入口起点経路、108/108 ホップの CGO 確立状態を復元した。プロセス単位 3/3 成功の二側 95% Wilson スコア区間は 43.85–100% であり、少数条件の成立確認であって配布環境一般の成功率推定ではない。

表 9　Tor Project 公式 Debian パッケージ版の評価

条件	プロセス	回路と入口順	CGO ホップ
Tor 0.4.9.17、ストリップ済み実行ファイル、対応する `tor-dbgsym`	3/3	36/36	108/108

外部デバッグシンボルを与えない条件では、解析器は終了状態 3 で型情報の欠如を報告し、回路を出力しなかった。異なる Build ID のデバッグファイルを配置した条件では、GDB が当該ファイルを読み込まず、同じく回路を出力しなかった。実行ファイルとコアダンプの対応検査では、正しい配布実行ファイルは比較可能な 368,640 バイトの 95.57% が一致して通過した。別ビルドの Tor 0.4.9.16 は 39.62% だけが一致し、解析前に拒否された。したがって、配布実行ファイルだけでは本手法を適用できないが、対応する外部デバッグシンボルを保全できれば、自己ビルド時と同じ E1–E3 の判定を実行できる。

一方、明示経路でない ambient 回路では、0.4.9.3-alpha の 3 起動で CGO 133 ホップ、TOR1 91 ホップ、0.4.9.9 で CGO 132、TOR1 93 だった。修正後の 0.4.9.16 では CGO 438、TOR1 12 である。この分布差は、0.4.9.12 で修正された「最終ホップ以外で CGO を交渉しない」条件と整合する [6]。ただし ambient 回路はホップ選択と長さを固定していないため、この値をネットワーク全体の普及率とは解釈しない。表 8 の 432/432 は、CGO が実体化した既知回路を各版の解析器が復元できるかという条件付き評価である。

LTO ビルドでは、DWARF 上の所有リスト位置を GDB から直接取得できなかった。ELF ローカルシンボルと PIE ロードバイアスを用いる代替処理により 12/12 回路、36/36 CGO ホップを復元した。QEMU 上の x86_64 ゲストコアでも 12/12 回路、36/36 ホップが一致した。

確立状態 12/12、構築中 8/8、正常終了後に所有リストから不在 6/6 を分類した。構築中の 16 ホップは列挙値だけを見ると TOR1 であったが、暗号状態は未初期化だった。本手法は方式を判定不能とし、TOR1 が確立したとは報告しなかった。

完全な ELF コア内の 4 KiB ページをゼロ化した 8 条件では、偽の構造的有効候補は 0 件だった。この評価は真のページ欠損ではなく、ゼロ化した擬似欠損である。読取りの成否と列挙完了を検査しない先行試作は、所有リストの `smartlist` をゼロ化した条件をエラーなしの 0 回路と報告した。この観測を受け、最終手法は所有リストを全要素読めない限り E1 を成立させない。

さらに、所有リストの `smartlist` を含むヒープ `PT_LOAD` と、所有リスト変数を含む実行ファイルのデータ `PT_LOAD` をそれぞれコアのプログラムヘッダから除外し、実際に読取り不能とした 2 条件を評価した。両条件とも `owner_list_unavailable` と読取りアドレスを報告し、有効回路を出力しなかった。

実行ファイル照合では、対応 ELF、同版 LTO、異版の 3 条件を用いた。対応 ELF は写像バイト 84.51% 一致で通過し、同版 LTO と異版は 8.71%、9.58% で専用の不一致エラーとなった。型解決失敗は 0 回路として扱わず、実行ファイル不一致として停止した。

約 731 MiB の完全なコアダンプに対する解析時間中央値は、制御回路 0/12/48/95 で 0.142/0.164/0.223/0.306 秒だった。最大 RSS は 57,404–60,224 KiB である。取得による停止時間は 1.33–1.47 秒、ハッシュ計算は 0.92–1.04 秒だった。

8.2　コアダンプ取得後の通信を復元した

クライアントのコアダンプから、制御下 12/12 ストリームの宛先、Control 回路 ID、入口起点経路を結合し、偽陽性・偽陰性はいずれも 0 だった。同じ所有回路から抽出した現行 CGO 状態へ、コアダンプ取得後に生成された次のリレーセルを入力した。手順固定後の独立 5 起動すべてでホップ 2 を認識し、493/493 バイトの本文が評価用平文と一致した。

負対照を CGO 状態全体へ拡張した。J 側 AES 鍵、J 側 POLYVAL 鍵、S 側 AES 鍵、S 側 POLYVAL 鍵、nonce、T'、ホップ順の 7 分類について各 1,024 状態、合計 7,168 状態を同じ実セルへ適用し、誤認識は 0 件だった。これらは決定論的対照であり、全誤状態からの無作為標本ではないため、偽陽性率の信頼区間へ一般化しない。継続通信中の任意時点取得では、3 プロセスから各 10 スナップショットを取り、CGO 状態抽出は 30/30 で成功した。ただし同一プロセス内のスナップショットは相関する。全 10 スナップショット成功を 1 成功と数える保守的集計は 3/3 であり、95% 区間は 43.85–100% である。

復号処理へ渡した入力は、取得時点が対応するクライアントコアダンプ、検証済み実行ファイルと型情報、入口リンク PCAP だけである。Control Protocol 由来の正解回路 ID と動的フック由来の正解セルは別系統に置き、復号用入力集合にも含めなかった。

保存済みの独立 10 取得物を、E1–E3 を後段の必須条件とする一連の処理で再解析した。全取得物で所有リストを完全列挙し、E3 を通過した全ホップ CGO 回路だけを対象回路一覧へ記録した。TLS 秘密抽出器と CGO 秘密抽出器は、この同一一覧に含まれる回路以外を処理しない。10/10 で E1、E2、E3 が連続して成立した。

迂回可能性の負の対照として、E3 回路が存在しない構造解析結果、対象回路一覧なし、回路アドレスを改変した一覧を与えた。一覧生成器、CGO 抽出器、TLS 抽出器は 4 条件すべてで非 0 終了し、秘密値を出力しなかった。

各取得物では OpenSSL 状態から暗号スイート `TLS_AES_256_GCM_SHA384` を直接取得し、コアダンプ内の書込みレコード番号 7 に対応する TLS 1.3 レコードを 1 件認証復号した。対象入口回路 ID を持つ 7 セルを、Control Protocol 由来の選択子を使わずに列挙した。

全候補へ同じ取得時点の CGO 状態の複製を適用した結果、各試行で認証に成功した候補は厳密に 1 件であり、その宛先はホップ 2 だった。処理終了後にだけ正解を読む評価器で、認証候補が正解候補と一致すること、PCAP 由来 509 バイトと動的正解、復元本文 493 バイトと正解平文が一致することを確認した。以上の E1–E5 は 10/10 で成立した（表 10）。

表 10　TLS レコードと CGO 層を順に復号した段階別結果

検証段階	結果	成功条件
E1 所有	10/10	所有リストを完全列挙
E2 構造	10/10	magic、index、入口閉路が整合
E3 暗号状態	10/10	活動中の全ホップ CGO 回路を対象回路一覧へ記録
E4 TLS とセル	10/10	レコード番号 7 で各 1 レコード、各 7 セル
E5 CGO 本文	10/10	認証候補が各 1 件、ホップ 2、本文 493/493 バイト

独立単位を Tor プロセスとすると、固定条件下の反復成功率 10/10 の二側 95% Wilson 区間は 72.25–100% である。停止時点をセルや TLS レコードの境界に合わせない追加実験も 6/6 で E1–E5 へ到達した。2 ホップ 3 起動では CGO 認識ホップ 1、3 ホップ 3 起動ではホップ 2 となった。コアダンプから得た TLS 書込みレコード番号は 17、23、29、37、53、68 と全試行で異なり、各番号で TLS 1.3 レコード 1 件を認証した。各試行で得た 7 セル候補のうち CGO 認証に成功したのは一意の 1 件であり、その後に確認した評価用マーカーも一致した。復元成功 6/6 の二側 95% Wilson スコア区間は 60.97–100% である。

この追加実験では Control Protocol 回路 ID、動的フックのセル、正解ホップ、正解平文を復号処理へ渡していない。したがって、固定レコード番号 7、固定ホップ 2、ストリーム停止によってだけ 10/10 が成立した可能性は棄却できた。ただし 1 ホップの一般 SOCKS ストリームは Tor が当該回路を通常の出口回路として使用せず、ホップ 0 を宛先とする通信復号は評価できなかった。誤鍵、別起動秘密、E3 不成立、対象回路一覧の改変、混在回路の各条件では、平文を出力しないことを確認した。一方、停止時点を選ばなかった 6 試行は全件成功したため、この 6 試行内に不成立例はなかった。

TLS 秘密を含まない PCAP だけを与えた対照では、118 パケット、TCP ペイロード 78 セグメントを再構成しても、既知セル、平文、合成識別子のいずれも回収できなかった。

TLS 側の負対照では、クライアント側通信秘密の 1 ビット反転と、別起動の TLS 秘密はいずれも認証レコード 0 件で停止した。一方、同じ接続秘密のまま書込みレコード番号を 7 から 8 へ変えると、後続の番号 8 のレコード自体は正しく認証された。AEAD 認証はレコードの暗号学的正しさを示すが、それが取得直後のレコードであることまでは示さない。この結果から、E4 はコアダンプ内レコード番号の非改変性と図 3 の取得順序を必要条件とする。

この結果は、メモリだけ、または PCAP だけで復号したという主張ではない。時点が一致する両方の証拠を、回路所有関係により結合した結果である。

8.3　時間差は構造対応より先に暗号状態を失わせた

100 MB のストリームを 12 回路で転送中に、クライアントと 3 リレーを順次取得した。実測 2.297、2.663、3.168 秒の取得差でも、リンク列は 36/36 ホップで対応した。一方、nonce と T' の完全一致は 0/36 だった。回路構造は秒単位で持続しても、暗号状態はセルごとに進むためである。

また、セル処理直後の現行状態を直前の前方向セルへ適用した 3 回は、すべて認識ホップが −1 となり、平文も一致しなかった。この結果は図 1 の時間境界を実 Tor で確認したものである。

表 11　成立した復元と成立しなかった復元

問い	結果	解釈
所有回路・入口起点経路	144/144	構造上の対応を再現
コアダンプ取得後の次セル	5/5	現行 CGO 状態が利用可能
コアダンプ＋ PCAP	10/10	TLS レコードからリレー本文まで復号
ストリーム非停止・停止時点非選択	6/6	TLS 番号 6 種、宛先ホップ 1/2 で E5
混在回路の E4 投入	0/10	E3 で報告し CGO 専用後段から除外
現行状態＋直前セル	0/3	過去状態を一般には逆算不可
順次取得した両端状態	0/36	瞬時照合には同時停止が必要
誤 CGO 状態の宛先誤認	0/7168	7 分類の決定論的負対照

9　抽出状態が与える攻撃能力の境界

メモリから暗号状態を抽出できても、それだけで遠隔攻撃や匿名性破壊が可能になるとは限らない。この脅威境界を確認するため、Pillwax が示した後方向 CGO に対する IPR+ 再同期識別攻撃を C Tor 0.4.9.16 で再現し、実際の 3 ホップ回路でどこまで進むかを調べた [26]。これは提案手法とは別の脆弱性主張ではなく、抽出状態から外挿してはならない能力を測る境界実験である。

単一中間層では、改変セルの左側タグを保持したまま右側を 1 ビット変更すると、そのセルは認識されないが、クライアント側とリレー側の対象層は再同期した。次の正常セルは 509/509 バイトが一致した。したがって、形式的な再同期性は実装にも存在する。

しかし、3 ホップ処理では外側の対象層だけが再同期し、改変セルを処理した内側 2 層の T' はリレー側と不一致になった。回路を仮に維持しても、次の正常な出口セルは認識されなかった。同じ 2 セル列は Arti 0.48.0 でも `BadCellAuth` として拒否された [27]。さらに、隔離 Chutney 網で入口リレーがセルを 1 回だけ改変した 3 回の独立起動では、3/3 でクライアントが対象回路を閉じた。95% Wilson 区間は 43.85–100% であり、小標本である。

以上から、既知の形式的攻撃は実装で再現できるが、本評価では鍵なし復号、セル偽造、匿名性破壊へ発展しなかった。全層処理による内側状態の不一致と、最初の認証失敗で回路を閉じる処理が、2 回目の問い合わせを妨げた。本研究はこれを新規脆弱性として主張しない。

10　考察

10.1　実務上の意味と能力境界

本手法が直接答えられる問いは具体的である。取得時点に Tor が所有していたクライアント起点回路、その Control 回路 ID、入口からのリレー順、ホップごとの CGO/TOR1、方向別 CGO 状態の初期化と更新有無を復元できる。ストリーム構造が残っていれば、SOCKS 宛先を所属回路と経路へ結び付けられる。コアダンプ取得時点をまたぐ入口 PCAP があれば、TLS レコードから Tor セルを取り出し、CGO を処理してリレー本文を回収できる。復元単位は 493 バイトの Tor リレーメッセージ本体である。DATA メッセージにアプリケーションバイトが含まれる場合でも、HTTPS や Onion Service 上位プロトコルの暗号文は別途復号しなければならない。

この情報は、侵害端末が調査時点にどの宛先へ接続し、どの回路を使い、どの暗号方式を実体化し、取得直後にどのリレーメッセージを送ったかを検証する用途に向く。通常ログに宛先や本文がなく、Control Port へ後から接続できない場合でも、取得済み証拠から再検証できる点が実務上の利点である。

ただし、ローカルホスト側の平文 PCAP がすでに取得され、目的がアプリケーション内容の確認だけなら、表 2 のとおり先にその方法を使うべきである。本手法が追加する価値は、平文らしい断片の発見ではなく、終了後の取得物から所有回路、入口順、ホップ方式、TLS レコード、CGO 認証済みリレーメッセージを同じ時点へ帰属させる点にある。実環境へ導入する際は、(1) 入口リンク PCAP の継続取得、(2)Tor 本体・共有ライブラリ・外部デバッグシンボルの保全、(3) 短時間のプロセス停止を伴う整合したコアダンプの取得、(4) 秘密ファイルのアクセス制御を取得計画へ含める必要がある。

合成送信元 `127.0.0.2` と合成識別子を用いた評価では、送信元、ストリーム、宛先、回路 ID、3 ホップ経路、復号内容中の識別子に関する 8 検査がすべて一致した。これは端点メモリ侵害下で匿名性メタデータが失われる仕組みを示す。ただし、実在人物の同定、接続元 IP の外部帰属、第三者通信の観測は行っていない。

TLS レコードと CGO 層の復号は、CGO の鍵を暗号解析で推定した結果ではない。端点メモリに正規の TLS 秘密と CGO 状態が存在し、それを Tor の所有関係から正しい回路へ帰属させた結果である。従って脅威モデルは端点メモリ読取りを含む。PCAP だけを持つ受動観測者には適用できない。

この区別はインシデント報告で重要である。「Tor を復号した」とだけ書けば、遠隔から Tor 全体を破れるように読める。本研究が示したのは、取得時点の端点状態を用いた、対象限定・時間限定の復号である。

暗号状態を両端で比較する場合、対象プロセスを先に同時停止し、その後でメモリを保存すべきである。順次取得では数秒で状態がずれる。一方、クライアント単体のコアダンプと同時刻 PCAP から、そのコアダンプ以後のセルを処理する場合は、両端メモリの同時取得を必要としない。必要なのは、コアダンプ内の TLS/CGO 状態と、PCAP 内セルの順序が一致することである。

現場では次の順序が適切である。まずパケット取得を継続したまま Tor を停止する。次にコアダンプ、実行ファイル、外部デバッグシンボル、Build ID、時刻、ハッシュを保存する。解析時は読み取りの成否と列挙の完了状態を確認し、不完全な 0 件を「不存在」と報告しない。

11　限界

第一に、TLS レコードと CGO 層を連続して復号する中心評価は Debian arm64、Tor 0.4.9.16、OpenSSL 3.0.22、`TLS_AES_256_GCM_SHA384` に限られる。x86_64 の構造復元は QEMU ゲストコアで確認したが、ネイティブ x86_64 で TLS から CGO までを通す評価は未実施である。OpenSSL のマイナーバージョンやプロバイダーが変われば、内部プロファイルを再検証する必要がある。

第二に、10/10 成功は固定条件での反復性を示し、停止時点をセルや TLS レコードの境界に合わせなかった 6/6 試行は、TLS レコード番号と宛先ホップへの一般化を補う。ただし、一般環境での成功率を精密に推定するものではない。二側 95% Wilson スコア区間はそれぞれ 72.25–100%、60.97–100% と広い。追加試行はレコード番号 17–68 と認識ホップ 1/2 を含むが、ホップ 0、異なる暗号スイート、CGO/TOR1 混在回路を層ごとに復号する処理は未評価である。混在回路は E3 で分類し、10/10 回路を CGO 専用復号処理へ渡さなかった段階に留まる。

中心 10 試行ではストリームを一時停止し、Tor を `SIGSTOP` してからコアダンプを取得した。追加 6 試行はストリームを止めず、停止時点をセルや TLS レコードの境界に合わせなかったが、`gcore` によるメモリ複写中のプロセス停止は残る。実行を止めないライブメモリ取得や、命令途中で不整合なページを収集する取得器は評価していない。したがって本結果は「通常通信中でも整合したスナップショットを取得すれば復元できる」ことを示すが、非原子的なメモリ取得一般を保証しない。

第三に、KeyUpdate、欠損 PCAP、TCP 部分重複、IP 断片化、IPv6、pcapng、後方向、複数連続セルの長時間再生を評価していない。実装の TCP 再構成は同一番号の完全再送だけを重複排除する。現行状態から直前セルを復号できなかったが、古い状態の別コピーやメモリアロケータの残留領域が残る可能性を一般には否定できない。

第四に、配布パッケージ版の評価は Tor Project 公式 Debian 12 arm64 パッケージ一種類と、同じリポジトリが配布する外部デバッグシンボルに限られる。3/3 起動で E1–E3 を確認したが、配布パッケージ版を使った TLS および CGO 復号、他のディストリビューション、外部デバッグシンボルを取得できない現場資料は評価していない。した

がって、任意の配布 Tor へそのまま適用できるとは主張しない。

第五に、C Tor 向けの型・所有関係を使用しており、Arti の Rust 型へそのまま適用できない。C Tor と Arti の CGO 逐次状態が正常操作列で一致することは確認したが、Arti では Rust の所有関係に対応する解析器が別途必要である。

第六に、文献調査は主要仕様、査読論文、実装、学位論文を対象としたスコーピングレビューである。未公開ツールや非英語文献を含む系統的レビューは完了していない。このため「世界初」とは主張しない。

最後に、本研究は管理下の隔離網と合成識別子だけを使用した。人物同定、広域の受動監視、遠隔からの匿名性破壊を実証していない。これらを本結果から外挿してはならない。

12　結論

本研究は、Tor の暗号状態を「鍵らしいバイト列」として探すのではなく、Tor 自身の所有関係から回路、入口、ホップ、TLS、CGO をたどるメモリフォレンジック手法を示した。複数の C Tor 0.4.9 系自己ビルドで 144/144 回路と 432/432 ホップを復元した。Tor Project 公式 Debian パッケージ版でも 36/36 回路と 108/108 ホップを復元した。取得時点が対応するクライアントコアダンプと PCAP による TLS レコードおよび CGO 層の復号は、ストリームを一時停止した 10/10 独立起動と、停止時点をセルや TLS レコードの境界に合わせなかった 6/6 独立起動で再現した。追加 6 試行では TLS 書込みレコード番号 6 種と宛先ホップ 1/2 へ条件を広げた。所有関係を検証した E3 対象回路一覧を TLS/CGO 抽出器の必須入力とすることで、E1 から E5 までを連続して成立させた。復号処理から評価用正解を物理的に分離し、各起動で Control Protocol 由来の正解回路 ID なしに 7 候補を列挙した後、CGO 認証により一意の 1 候補を確定した。CGO/TOR1 混在 10 回路は E3 で分類し、CGO 専用の後段へは 0 件を投入した。これにより、保存 PCAP、TLS レコード、Tor 回路、CGO ホップ、493 バイトのリレー本文の対応関係を検証できる。

同時に、成立範囲も明確になった。現行 CGO 状態は過去通信を包括的に開く鍵ではない。直前セルは 3/3 で復号できず、活動中の順次取得では状態一致が 0/36 になった。既知の後方向再同期攻撃も、3 ホップ処理と回路終了によって、鍵なし復号や匿名性破壊には至らなかった。

一般化には、ネイティブ x86_64、異なる OpenSSL プロバイダー、TLS KeyUpdate、欠損 PCAP、後方向、Arti の所有構造に対する評価が残る。拡張時にも、復号範囲だけでなく、説明可能な証拠条件・時点・回路を明示する必要がある。

データと実装の利用可能性

補足資料には、解析器、実験スクリプト、環境情報、ソースコミット、SHA-256 一覧を収録する。主要集計は、自己ビルド版の構造復元を `results/generalization/summary-arm64-replicated.json`、公式 Debian パッケージ版を `results/generalization/summary-official-package-0.4.9.17.json`、E1–E5 の 10 試行を `results/open-circuit-impact/independent-v3/summary.json`、停止時点を選ばなかった 6 試行を `results/uncontrolled-pcap/summary.json`、混在回路を CGO 専用復号処理へ渡さない評価を `results/real-tor-mixed/mixed-e3-admission.json`、CGO 負の対照を `results/open-circuit-impact/negative-state-matrix-v2/summary.json` へ固定した。対応表と再生成可能範囲は `paper/ARTIFACTS_ja.md` に記載する。

補足資料の公開先は https://github.com/quantumshiro/tor-cgo-analysis であり、評価対象を固定するリリースタグは `v0.0.3` である。公開用アーカイブ paper/tor-cgo-forensics-public-supplement-v5.tar.gz は固定リストから再生成できる。公開対象は秘密値と第三者情報を含まない集計、隔離網設定、再生成手順である。

生の TLS 秘密、CGO 鍵、コアダンプ、復号済みレコードは公開対象から除外するため、公開物だけで保存済み取得物を完全に再解析することはできない。公開物では、集計の監査と隔離網での再取得手順を提供する。タグが指すコミット SHA は `gitrev-list-n1v0.0.3` で検証できる。

倫理的配慮

全実験は管理下の隔離ネットワークで実施し、第三者の Tor 通信を観測・収集していない。送信元、宛先、識別子は合成値を使用した。本手法は、調査権限のある端末から取得した証拠の解析を想定する。実環境へ適用する場合は、法的権限、証拠保全、秘密情報の最小取得、アクセス制御を別途確認する必要がある。

著者の貢献

Hiro Nakanishi：Conceptualization、Methodology、Software、Validation、Formal Analysis、Investigation、Data Curation、Writing – Original Draft、Writing – Review & Editing、Visualization。

利益相反

申告すべき利益相反はない。

研究資金

本研究は外部資金の提供を受けていない。

生成 AI の利用

論文の構成整理、日本語表現の推敲、LaTeX 記述の補助に生成 AI を使用した。研究上の主張、実験値、引用、コード、結論は著者が一次資料と保存済み実験結果に照らして確認した。生成 AI は実験の実施主体、著者、引用元として扱わない。

参考文献

[1] Roger Dingledine, Nick Mathewson, and Paul Syverson. “Tor: The Second-Generation Onion Router”. In: *Proceedings of the 13th USENIX Security Symposium*. 2004, pp. 303–320. URL: https://www.usenix.org/conference/13th-usenix-security-symposium/tor-second-generation-onion-router.

[2] Nick Mathewson. *Proposal 359: Counter Galois Onion, Redux*. Tor Proposal 359. The Tor Project, 2024. URL: https://spec.torproject.org/proposals/359-cgo-redux.html (visited on 11/14/2026).

[3] Jean Paul Degabriele, Alessandro Melloni, Jean-Pierre Münch, and Martijn Stam. *Counter Galois Onion (CGO) for Tor: Fast Non-Malleable Onion Encryption*. Cryptology ePrint Archive, Paper 2025/583. 2025. URL: https://eprint.iacr.org/2025/583.

[4] The Tor Project. *C Tor Source, Release 0.4.9.16, Commit c51547ca15ea665950dd154e160051c57f5719db*. 2026. URL: https://gitlab.torproject.org/tpo/core/tor/-/commit/c51547ca15ea665950dd154e160051c57f5719db (visited on 11/14/2026).

[5] The Tor Project. *Stable Release 0.4.9.5*. Feb. 12, 2026. URL: https://forum.torproject.org/t/stable-release-0-4-9-5/21227 (visited on 11/14/2026).

[6] The Tor Project. *Tor Stable 0.4.9.12 Release Announcement*. Sept. 8, 2026. URL: https://www.mail-archive.com/tor-announce@lists.torproject.org/msg00324.html (visited on 11/14/2026).

[7] The Tor Project. *Tor Control Protocol Specification*. 2026. URL: https://spec.torproject.org/control-spec/ (visited on 11/14/2026).

[8] Jean Paul Degabriele and Martijn Stam. “Untagging Tor: A Formal Treatment of Onion Encryption”. In: *Advances in Cryptology – EUROCRYPT 2018*. 2018, pp. 259–293. DOI: 10.1007/978-3-319-78372-7_9.

[9] Jean Paul Degabriele, Alessandro Melloni, and Martijn Stam. *Secure Onion Encryption and the Case of Counter Galois Onion*. Cryptology ePrint Archive, Paper 2025/2017. 2025. URL: https://eprint.iacr.org/2025/2017.

[10] Malak Alfosail and Peter Norris. “Tor Forensics: Proposed Workflow for Client Memory Artefacts”. In: *Computers & Security* 106 (2021), p. 102311. DOI: 10.1016/j.cose.2021.102311.

[11] Matt Muir, Petra Leimich, and William J. Buchanan. “A Forensic Audit of the Tor Browser Bundle”. In: *Digital Investigation* 29 (2019), pp. 118–128. DOI: 10.1016/j.diin.2019.03.009.

[12] Muhammad Raheel Arshad, Mehdi Hussain, Hasan Tahir, Sana Qadir, Faraz Iqbal Ahmed Memon, and Yousra Javed. “Forensic Analysis of Tor Browser on Windows 10 and Android 10 Operating Systems”. In: *IEEE Access* 9 (2021). DOI: 10.1109/ACCESS.2021.3119724.

[13] The Tor Project. *Privacy Analysis of Tor's In-Memory Statistics*. Tech. rep. 2017-04-001. The Tor Project, 2017. URL: https://research.torproject.org/techreports/privacy-in-memory-2017-04-28.pdf.

[14] Carsten Maartmann-Moe, Steffen E. Thorkildsen, and André Årnes. "The Persistence of Memory: Forensic Identification and Extraction of Cryptographic Keys". In: *Digital Investigation* 6 (2009), S132–S140. DOI: 10.1016/j.diin.2009.06.002.

[15] J. Alex Halderman, Seth D. Schoen, Nadia Heninger, William Clarkson, William Paul, Joseph A. Calandrino, Ariel J. Feldman, Jacob Appelbaum, and Edward W. Felten. "Lest We Remember: Cold Boot Attacks on Encryption Keys". In: *Proceedings of the 17th USENIX Security Symposium*. 2008, pp. 45–60. URL: https://www.usenix.org/legacy/events/sec08/tech/full_papers/halderman/halderman.pdf.

[16] Florent Moriconi, Olivier Levillain, Aurélien Francillon, and Raphael Troncy. "X-Ray-TLS: Transparent Decryption of TLS Sessions by Extracting Session Keys from Memory". In: *Proceedings of the 19th ACM Asia Conference on Computer and Communications Security*. 2024. DOI: 10.1145/3634737.3637654. URL: https://doi.org/10.1145/3634737.3637654.

[17] Daniel Baier, Alexander Basse, Jan-Niclas Hilgert, and Martin Lambertz. "TLS Key Material Identification and Extraction in Memory: Current State and Future Challenges". In: *Forensic Science International: Digital Investigation* 49 (2024), p. 301766. DOI: 10.1016/j.fsidi.2024.301766. URL: https://doi.org/10.1016/j.fsidi.2024.301766.

[18] Daniel Baier and Martin Lambertz. "All Your TLS Keys Are Belong to Us: A Novel Approach to Live Memory Forensic Key Extraction". In: *Forensic Science International: Digital Investigation* 54 (2025), p. 301975. DOI: 10.1016/j.fsidi.2025.301975. URL: https://doi.org/10.1016/j.fsidi.2025.301975.

[19] Erik Hjelmvik. *Hands-on Analysis of Network Packets Carved from Memory and PCAP Analysis of Unencrypted Tor Traffic*. DFRWS EU 2026 Workshop. 2026. URL: https://dfrws.org/dfrws-eu-2026-workshop-network-packets-memory-tor-traffic/ (visited on 11/14/2026).

[20] Eric Rescorla. *The Transport Layer Security (TLS) Protocol Version 1.3*. RFC 8446. RFC Editor, Aug. 2018. DOI: 10.17487/RFC8446. URL: https://www.rfc-editor.org/rfc/rfc8446.html (visited on 11/14/2026).

[21] Free Software Foundation. *GDB Manual: Core Files in Python*. 2026. URL: https://sourceware.org/gdb/current/onlinedocs/gdb.html/Core-Files-In-Python.html (visited on 11/14/2026).

[22] DWARF Debugging Information Format Committee. *DWARF Debugging Information Format Version 5*. 2017. URL: https://dwarfstd.org/dwarf5std.html (visited on 11/14/2026).

[23] OpenSSL Project. *OpenSSL 3.0.22 Source*. 2026. URL: https://github.com/openssl/openssl/tree/openssl-3.0.22 (visited on 11/14/2026).

[24] The Tor Project. *Chutney Source, Commit 6036fae4bd94c53e1f72967b9fb6c10d9c7fbd7e*. 2026. URL: https://gitlab.torproject.org/tpo/core/chutney/-/commit/6036fae4bd94c53e1f72967b9fb6c10d9c7fbd7e (visited on 11/14/2026).

[25] The Tor Project. *Debian 12 arm64 Package Index: tor and tor-dbgsym 0.4.9.17*. 2026. URL: https://deb.torproject.org/torproject.org/dists/bookworm/main/binary-arm64/Packages (visited on 11/14/2026).

[26] Wolfgang Pillwax. "Improving Onion Routing in Tor". MA thesis. TU Wien, 2026. DOI: 10.34726/hss.2026.129611.

[27] The Tor Project. *Arti Source, Commit 5a39d48e31a2a0417d8850e21e9b8fa002ed1be5*. 2026. URL: https://gitlab.torproject.org/tpo/core/arti/-/commit/5a39d48e31a2a0417d8850e21e9b8fa002ed1be5 (visited on 11/14/2026).